



Big-Step 증명나무 합성으로 분석기 골탕먹이기

정원준

2026년 5월 15일

ROPAS@SNU

목차

정적 분석기의 부정확성

프로그램 합성과 증명나무 합성

증명나무 합성 디테일

현재까지 결과

앞으로 할 일

정적 분석기의 부정확성

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?
- 프로그램을 돌려보지 않고 미리 알 수는 없을까?

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?
- 프로그램을 돌려보지 않고 미리 알 수는 없을까?
- 완벽하게 알 수는 없음!

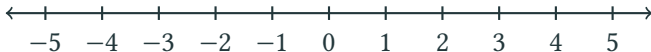
정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?
- 프로그램을 돌려보지 않고 미리 알 수는 없을까?
- 완벽하게 알 수는 없음!
- “Rice’s Theorem”

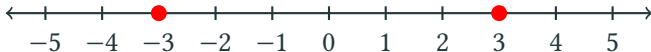
넓게 어림잡기

```
if x > 0:
    x = 3
else:
    x = -3
print(x)
```



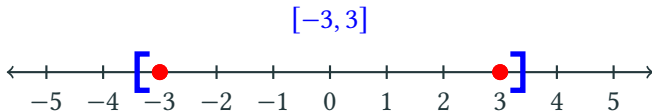
넓게 어림잡기

```
if x > 0:
    x = 3
else:
    x = -3
print(x)
```



넓게 어림잡기

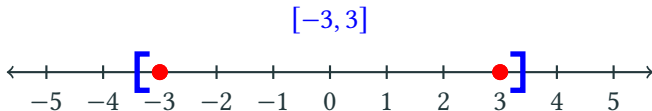
```
if x > 0:
    x = 3
else:
    x = -3
print(x)
```



공격 프로그램의 예시

공격 프로그램의 예시

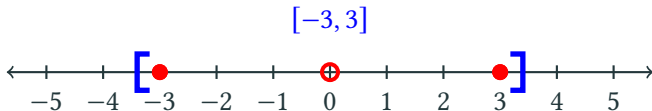
```
if x > 0:
    x = 3
else:
    x = -3
print(x)
```



공격 프로그램의 예시

```
if x > 0:
    x = 3
else:
    x = -3
y = 10 / x
```

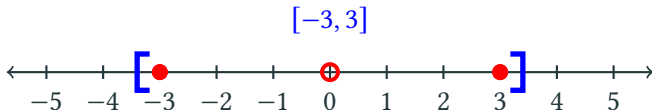
false alarm 발생



공격 프로그램의 예시

```
if x > 0:
    x = 3
else:
    x = -3
y = 10 / x
```

false alarm 발생



이런 공격을 일반적인 분석기에 대해서 하는 것이 목표

공격의 기대효과

공격의 기대효과

- 분석기의 약점을 파악하고 강화하는 가이드라인으로 사용

공격의 기대효과

- 분석기의 약점을 파악하고 강화하는 가이드라인으로 사용
- 분석이 잘 안 되는 코드 조각을 삽입해서 난독화로 사용

프로그램 합성과 증명나무 합성

프로그램 합성

- 주어진 성질을 만족하는 프로그램을 자동으로 생성!

프로그램 합성

- 주어진 성질을 만족하는 프로그램을 자동으로 생성!
- “입력이 1, 2, 3, 4일 때 각각 1, 3, 6, 10이 나오는 프로그램을 찾아”

프로그램 합성

- 주어진 성질을 만족하는 프로그램을 자동으로 생성!
- “입력이 1, 2, 3, 4일 때 각각 1, 3, 6, 10이 나오는 프로그램을 찾아”
- “입력이 x 이고 출력이 y 일 때, $y^2 \leq x < (y + 1)^2$ 가 되는 프로그램을 찾아”

프로그램 합성

- 정답이 있는 문제의 경우

프로그램 합성

- 정답이 있는 문제의 경우
- bottom-up enumeration을 통해 유한 시간 안에 끝남을 보장

프로그램 합성

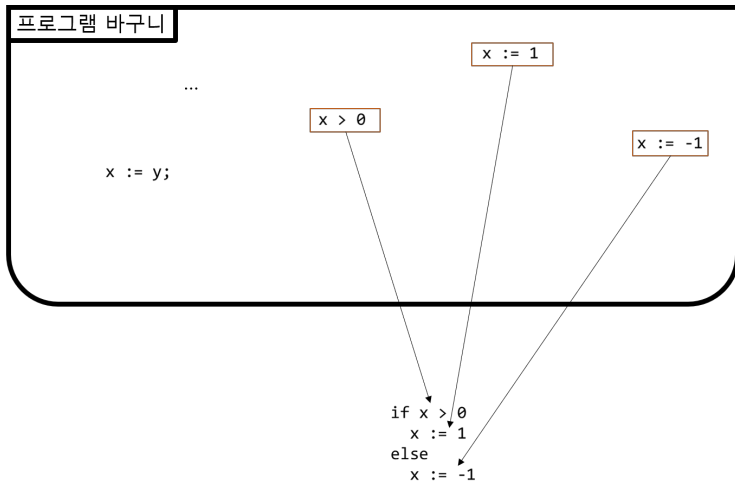
- 정답이 있는 문제의 경우
- bottom-up enumeration을 통해 유한 시간 안에 끝남을 보장
- 모든 프로그램을 빠짐없이 검사 → 여러 가지 트릭으로 시간 단축

프로그램 합성 예시

프로그램 바꾸니

```
x := 1  
...  
x > 0  
x := -1  
x := y;
```

프로그램 합성 예시

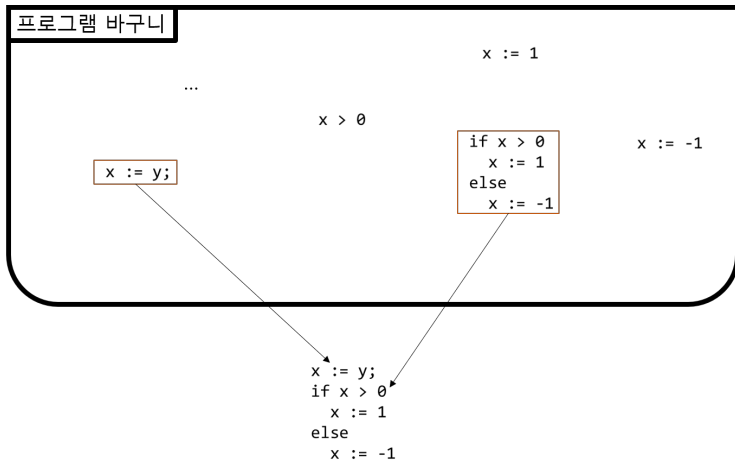


프로그램 합성 예시

프로그램 바꾸니

```
x := y;  
  
...  
  
x > 0  
  
x := 1  
  
if x > 0  
  x := 1  
else  
  x := -1  
  
x := -1
```

프로그램 합성 예시



프로그램 합성 예시

프로그램 바꾸니

```
...  
x := y;  
    if x > 0  
    x := 1  
    else  
    x := -1  
x := 1  
if x > 0  
  x := 1  
else  
  x := -1  
x := -1
```

분석기 골탕먹이기 문제

- “분석기가 false alarm을 내는 프로그램을 찾아”

분석기 골탕먹이기 문제

- “분석기가 false alarm을 내는 프로그램을 찾아”
- Rice's Theorem 덕분에 정답이 있음을 보장

분석기 골탕먹이기 문제

- “분석기가 false alarm을 내는 프로그램을 찾아”
- Rice's Theorem 덕분에 정답이 있음을 보장
- 그렇다면 전탐색으로 쉽게 유한 시간 안에 풀 수 있겠네?

분석기 골탕먹이기 문제

- “분석기가 false alarm을 내는 프로그램을 찾아”
- Rice's Theorem 덕분에 정답이 있음을 보장
- 그렇다면 전탐색으로 쉽게 유한 시간 안에 풀 수 있겠네?
- **No.** false alarm이 진짜 나는지 알 방법이 없음

증명나무 합성

- 프로그램의 “진짜” 실행 의미를 알 수 없다는 게 문제

증명나무 합성

- 프로그램의 “진짜” 실행 의미를 알 수 없다는 게 문제
- 어떻게 하면 프로그램을 합성할 때 실행 의미까지 같이 끌고 다닐 수 있을까?

증명나무 합성

- 프로그램의 “진짜” 실행 의미를 알 수 없다는 게 문제
- 어떻게 하면 프로그램을 합성할 때 실행 의미까지 같이 끌고 다닐 수 있을까?
- 프로그램의 실행 의미를 Big-Step으로 정의해서 그 증명나무를 합성해보자

증명나무 합성 예시

증명나무 바꾸니

...

$$\frac{\{\} \mid - 1 \Rightarrow 1}{\{\} \mid - x := 1 \Rightarrow \{x: 1\}}$$

$$\frac{\frac{\{x: 1\} \mid - x \Rightarrow 1 \quad \{x: 1\} \mid - 1 \Rightarrow 1}{\{x: 1\} \mid - (x < 1) \Rightarrow \emptyset}}$$

증명나무 합성 예시

증명나무 바꾸니

...

```
-----  
{ } |- 1 => 1  
-----  
{ } |- x := 1 => {x: 1}
```

```
-----  
{x: 1} |- x => 1    {x: 1} |- 1 => 1  
-----  
{x: 1} |- (x < 1) => 0
```

↓

```
-----  
{x: 1} |- x => 1    {x: 1} |- 1 => 1  
-----  
{x: 1} |- (x < 1) => 0  
-----  
{x: 1} |- while (x < 1) => {x: 1}  
      x := (x + 1)
```

증명나무 합성 예시

증명나무 바꾸니

...

```
-----  
{ } |- 1 => 1  
-----  
{ } |- x := 1 => {x: 1}
```

```
-----  
{x: 1} |- x => 1    {x: 1} |- 1 => 1  
-----  
{x: 1} |- (x < 1) => 0
```

```
-----  
{x: 1} |- x => 1    {x: 1} |- 1 => 1  
-----  
{x: 1} |- (x < 1) => 0  
-----  
{x: 1} |- while (x < 1) => {x: 1}  
      x := (x + 1)
```

증명나무 합성 예시

증명나무 바꾸니

...

$$\frac{\{x: 1\} \vdash x \Rightarrow 1 \quad \{x: 1\} \vdash 1 \Rightarrow 1}{\{x: 1\} \vdash (x < 1) \Rightarrow \emptyset}$$

$$\frac{\{ \} \vdash 1 \Rightarrow 1}{\{ \} \vdash x := 1 \Rightarrow \{x: 1\}}$$

$$\frac{\{x: 1\} \vdash x \Rightarrow 1 \quad \{x: 1\} \vdash 1 \Rightarrow 1}{\{x: 1\} \vdash (x < 1) \Rightarrow \emptyset}$$

$$\{x: 1\} \vdash \text{while } (x < 1) \Rightarrow \{x: 1\}$$

$$x := (x + 1)$$

$$\frac{\{ \} \vdash 1 \Rightarrow 1}{\{ \} \vdash x := 1 \Rightarrow \{x: 1\}}$$

$$\frac{\{x: 1\} \vdash x \Rightarrow 1 \quad \{x: 1\} \vdash 1 \Rightarrow 1}{\{x: 1\} \vdash (x < 1) \Rightarrow \emptyset}$$

$$\{x: 1\} \vdash \text{while } (x < 1) \Rightarrow \{x: 1\}$$

$$x := (x + 1)$$

$$\{ \} \vdash x := 1; \text{while } (x < 1) \Rightarrow \{x: 1\}$$

$$x := (x + 1)$$

증명나무 합성으로 분석기 골탕먹이기 문제 해결

- 증명나무가 존재하는 프로그램은 항상 유한하게 종료됨

증명나무 합성으로 분석기 골탕먹이기 문제 해결

- 증명나무가 존재하는 프로그램은 항상 유한하게 종료됨
- 또한 최종 environment는 singleton set이 됨

증명나무 합성으로 분석기 골탕먹이기 문제 해결

- 증명나무가 존재하는 프로그램은 항상 유한하게 종료됨
- 또한 최종 environment는 singleton set이 됨
- 따라서 분석기가 singleton set보다 큰 결과를 낸다면, false alarm이 발생함

증명나무 합성으로 분석기 골탕먹이기 문제 해결

- 증명나무가 존재하는 프로그램은 항상 유한하게 종료됨
- 또한 최종 environment는 singleton set이 됨
- 따라서 분석기가 singleton set보다 큰 결과를 낸다면, false alarm이 발생함
- 역시 Rice's Theorem 덕분에 유한 시간 안에 끝남이 보장됨

증명나무 합성 디테일

증명나무의 올바른

```
-----  
{} |- 1 => 1  
-----  
{} |- x := 1 => {x: 1}  
-----  
{} |- x := 1;      => {x: 1}  
   while (x < 1)  
     x := (x + 1)
```

```
-----  
{x: 1} |- x => 1    {x: 1} |- 1 => 1  
-----  
{x: 1} |- (x < 1) => 0  
-----  
{x: 1} |- while (x < 1)  => {x: 1}  
                x := (x + 1)
```

증명나무의 올바름

```
-----  
{} |- 1 => 1  
-----  
{} |- x := 1 => {x: 1}  
-----  
{x: 1} |- x => 1  
{x: 1} |- 1 => 1  
-----  
{x: 1} |- (x < 1) => 0  
-----  
{x: 1} |- while (x < 1) => {x: 1}  
           x := (x + 1)  
-----  
{} |- x := 1;  
      while (x < 1)  
      x := (x + 1)  
      => {x: 1}
```

증명나무의 올바름

```
-----  
{} |- 1 => 1  
-----  
{} |- x := 1 => {x: 1}  
-----  
{} |- x := 1; while (x < 1) x := (x + 1) => {x: 1}  
-----  
{x: 1} |- x => 1    {x: 1} |- 1 => 1  
-----  
{x: 1} |- (x < 1) => 0  
-----  
{x: 1} |- while (x < 1) x := (x + 1) => {x: 1}
```

증명나무의 올바름

```
-----  
{} |- 1 => 1  
-----  
{} |- x := 1 => {x: 1}  
-----  
{} |- x := 1;      => {x: 1}  
   while (x < 1)  
     x := (x + 1)
```

```
-----  
{x: 1} |- x => 1    {x: 1} |- 1 => 1  
-----  
{x: 1} |- (x < 1) => 0  
-----  
{x: 1} |- while (x < 1) => {x: 1}  
                x := (x + 1)  
-----
```

environment 제한

같은 프로그램이라도, environment에 따라서 증명나무가 달라짐

$$\begin{array}{lcl} \{x: 1\} & |- & x \Rightarrow 1 \\ \{x: 2\} & |- & x \Rightarrow 2 \\ \{x: 3\} & |- & x \Rightarrow 3 \\ \{x: 4\} & |- & x \Rightarrow 4 \\ & \vdots & \end{array}$$

environment 제한

같은 프로그램이라도, environment에 따라서 증명나무가 달라짐

$$\begin{array}{lcl} \{x: 1\} & |- & x \Rightarrow 1 \\ \{x: 2\} & |- & x \Rightarrow 2 \\ \{x: 3\} & |- & x \Rightarrow 3 \\ \{x: 4\} & |- & x \Rightarrow 4 \\ & \vdots & \end{array}$$

- 가능한 environment들을 제한해야 함

environment 제한

같은 프로그램이라도, environment에 따라서 증명나무가 달라짐

$$\begin{array}{lcl} \{x: 1\} & |- & x \Rightarrow 1 \\ \{x: 2\} & |- & x \Rightarrow 2 \\ \{x: 3\} & |- & x \Rightarrow 3 \\ \{x: 4\} & |- & x \Rightarrow 4 \\ & \vdots & \end{array}$$

- 가능한 environment들을 제한해야 함
- 유한 시간을 보장하려면, 가능한 environment를 점점 키워야 함

증명나무의 크기

bottom-up enumeration은 크기 순서로 진행됨

증명나무의 크기

bottom-up enumeration은 크기 순서로 진행됨

‘크기’를 정의하는 것이 중요

증명나무의 크기

bottom-up enumeration은 크기 순서로 진행됨

‘크기’를 정의하는 것이 중요

크기함수의 조건?

증명나무의 크기

bottom-up enumeration은 크기 순서로 진행됨

‘크기’를 정의하는 것이 중요

크기함수의 조건?

- 모든 증명나무는 유한한 크기를 가짐

증명나무의 크기

bottom-up enumeration은 크기 순서로 진행됨

‘크기’를 정의하는 것이 중요

크기함수의 조건?

- 모든 증명나무는 유한한 크기를 가짐
- 부분 증명나무의 크기를 더해 전체 증명나무의 크기를 구할 수 있으면 좋음

증명나무의 크기

bottom-up enumeration은 크기 순서로 진행됨

‘크기’를 정의하는 것이 중요

크기함수의 조건?

- 모든 증명나무는 유한한 크기를 가짐
- 부분 증명나무의 크기를 더해 전체 증명나무의 크기를 구할 수 있으면 좋음
- 한 크기에 대해, 해당하는 증명나무의 수는 유한해야 함

증명나무의 크기를 직관적으로 설정하면...

$$\frac{\{x: 1\} \mid - (x < 1) \Rightarrow \emptyset}{\{x: 1\} \mid - \text{while } (x < 1) \Rightarrow \{x: 1\} \quad x := (x + 1)}$$

증명나무의 크기를 직관적으로 설정하면...

$$\{x: 1\} \mid - (x < 1) \Rightarrow 0$$

$$\{x: 1\} \mid - \text{while } (x < 1) \Rightarrow \{x: 1\}$$
$$x := (x + 1);$$
$$x := (x + 1);$$

증명나무의 크기를 직관적으로 설정하면...

$$\{x: 1\} \mid - (x < 1) \Rightarrow \emptyset$$

$$\{x: 1\} \mid - \text{while } (x < 1) \Rightarrow \{x: 1\}$$
$$\begin{array}{l} x := (x + 1); \\ x := (x + 1); \\ x := (x + 1); \end{array}$$

증명나무의 크기를 직관적으로 설정하면...

$$\{x: 1\} \mid - (x < 1) \Rightarrow \emptyset$$

 $\{x: 1\} \mid - \text{while } (x < 1) \Rightarrow \{x: 1\}$

$$x := (x + 1);$$
$$x := (x + 1);$$
$$x := (x + 1);$$
$$x := (x + 1)$$

증명나무의 크기를 직관적으로 설정하면...

$$\{x: 1\} \vdash (x < 1) \Rightarrow 0$$

$$\{x: 1\} \vdash \text{while } (x < 1) \Rightarrow \{x: 1\}$$
$$\begin{array}{l} x := (x + 1); \\ x := (x + 1); \\ x := (x + 1); \\ x := (x + 1) \end{array}$$

같은 크기의 증명나무의 수가 무한해짐

증명나무의 크기를 직관적으로 설정하면...

$$\{x: 1\} \mid - (x < 1) \Rightarrow \emptyset$$

$$\{x: 1\} \mid - \text{while } (x < 1) \Rightarrow \{x: 1\}$$
$$\begin{array}{l} x := (x + 1); \\ x := (x + 1); \\ x := (x + 1); \\ x := (x + 1) \end{array}$$

같은 크기의 증명나무의 수가 무한해짐

당연히, 프로그램의 크기를 기준으로 해도 증명나무가 계속 커짐

(proof, prog) pair로 크기 정의

```
-----  
(1, 1) {} |- 1 => 1  
-----  
(2, 2) {} |- x := 1 => {x: 1}  
-----  
-----  
(1, 1) {x: 1} |- x => 1      (1, 1) {x: 1} |- 1 => 1  
-----  
-----  
(3, 3) {x: 1} |- (x < 1) => 0  
-----  
(8, 4) {x: 1} |- while (x < 1) => {x: 1}  
                    x := (x + 1)  
-----  
(11, 7) {} |- x := 1;      => {x: 1}  
                    while (x < 1)  
                    x := (x + 1)
```

(proof, prog) pair로 크기 정의

```
-----  
(1, 1) {x: 1} |- x => 1      (1, 1) {x: 1} |- 1 => 1  
-----  
-----  
(3, 3) {x: 1} |- (x < 1) => 0  
-----  
-----  
(2, 2) {x: 1} |- x := 1 => {x: 1}      (8, 4) {x: 1} |- while (x < 1) => {x: 1}  
                                     x := (x + 1)  
-----  
-----  
(11, 7) {x: 1} |- x := 1;           => {x: 1}  
                    while (x < 1)  
                    x := (x + 1)
```

크기 순서를 대각선($1, 1 \rightarrow 2, 1 \rightarrow 2, 2 \rightarrow 3, 1 \rightarrow \dots$)으로 잡으면

(proof, prog) pair로 크기 정의

```
-----  
(1, 1) {x: 1} |- x => 1    (1, 1) {x: 1} |- 1 => 1  
-----  
-----  
(3, 3) {x: 1} |- (x < 1) => 0  
-----  
-----  
(2, 2) {x: 1} |- x := 1 => {x: 1}    (8, 4) {x: 1} |- while (x < 1) => {x: 1}  
                                     x := (x + 1)  
-----  
-----  
(11, 7) {x: 1} |- x := 1;           => {x: 1}  
                      while (x < 1)  
                      x := (x + 1)
```

크기 순서를 대각선($1, 1 \rightarrow 2, 1 \rightarrow 2, 2 \rightarrow 3, 1 \rightarrow \dots$)으로 잡으면

- 모든 증명나무는 유한한 크기를 가짐? OK

(proof, prog) pair로 크기 정의

<pre> ----- (1, 1) {} - 1 => 1 ----- (2, 2) {} - x := 1 => {x: 1} ----- </pre>	<pre> ----- (1, 1) {x: 1} - x => 1 (1, 1) {x: 1} - 1 => 1 ----- (3, 3) {x: 1} - (x < 1) => 0 ----- (8, 4) {x: 1} - while (x < 1) => {x: 1} x := (x + 1) ----- (11, 7) {} - x := 1; => {x: 1} while (x < 1) x := (x + 1) </pre>
--	---

크기 순서를 대각선($1, 1 \rightarrow 2, 1 \rightarrow 2, 2 \rightarrow 3, 1 \rightarrow \dots$)으로 잡으면

- 모든 증명나무는 유한한 크기를 가짐? OK
- 부분 증명나무의 크기를 더해 전체 증명나무의 크기를 구할 수 있으면 좋음? OK

(proof, prog) pair로 크기 정의

----- (1, 1) {} - 1 => 1 -----	----- (1, 1) {x: 1} - x => 1 (1, 1) {x: 1} - 1 => 1 -----
----- (2, 2) {} - x := 1 => {x: 1} -----	----- (3, 3) {x: 1} - (x < 1) => 0 -----
	----- (8, 4) {x: 1} - while (x < 1) => {x: 1} x := (x + 1) -----
	----- (11, 7) {} - x := 1; => {x: 1} while (x < 1) x := (x + 1) -----

크기 순서를 대각선($1, 1 \rightarrow 2, 1 \rightarrow 2, 2 \rightarrow 3, 1 \rightarrow \dots$)으로 잡으면

- 모든 증명나무는 유한한 크기를 가짐? OK
- 부분 증명나무의 크기를 더해 전체 증명나무의 크기를 구할 수 있으면 좋음? OK
- 한 크기에 대해, 해당하는 증명나무의 수는 유한해야 함? OK

(proof, prog) pair로 크기 정의

-----	-----
(1, 1) {} - 1 => 1	(1, 1) {x: 1} - x => 1 (1, 1) {x: 1} - 1 => 1
-----	-----
(2, 2) {} - x := 1 => {x: 1}	(3, 3) {x: 1} - (x < 1) => 0
-----	-----
	(8, 4) {x: 1} - while (x < 1) => {x: 1} x := (x + 1)
-----	-----
	(11, 7) {} - x := 1; => {x: 1} while (x < 1) x := (x + 1)

크기 순서를 대각선(1, 1 → 2, 1 → 2, 2 → 3, 1 → ...)으로 잡으면

- 모든 증명나무는 유한한 크기를 가짐? OK
- 부분 증명나무의 크기를 더해 전체 증명나무의 크기를 구할 수 있으면 좋음? OK
- 한 크기에 대해, 해당하는 증명나무의 수는 유한해야 함? OK

실행되지 않는 코드 조각을 위해 일반 프로그램 또한 같이 합성해야 함

pruning 방법

- 프로그램 합성에서는 실행 의미가 같은 프로그램은 하나만 남기고 버려도 됨

pruning 방법

- 프로그램 합성에서는 실행 의미가 같은 프로그램은 하나만 남기고 버려도 됨
- 증명나무 합성에서는 실행 의미가 같더라도 막 제거하면 안 됨. 분석기가 분석을 다르게 할 수 있기 때문

pruning 방법

- 프로그램 합성에서는 실행 의미가 같은 프로그램은 하나만 남기고 버려도 됨
- 증명나무 합성에서는 실행 의미가 같더라도 막 제거하면 안 됨. 분석기가 분석을 다르게 할 수 있기 때문
- 그렇지만, 분석기를 ‘존중’하는 pruning은 가능

pruning 방법

- 프로그램 합성에서는 실행 의미가 같은 프로그램은 하나만 남기고 버려도 됨
- 증명나무 합성에서는 실행 의미가 같더라도 막 제거하면 안 됨. 분석기가 분석을 다르게 할 수 있기 때문
- 그렇지만, 분석기를 ‘존중’하는 pruning은 가능
- $((p; q); r) = (p; (q; r))$, $(x + y) = (y + x)$ 와 같이 실행해 보지 않고 항상 미리 알 수 있는 경우

현재까지 결과

첫 번째 공격

```
while 0
  x := 1;
  while 0
    x := 0
```

첫 번째 공격

```
while 0
  x := 1;
  while 0
    x := 0
```

분석기의 조건문 필터링이 안에 변수가 있을 때만 작동하고, 아예 true/false일 경우 체크를 못함

첫 번째 공격

```
while 0
  x := 1;
  while 0
    x := 0
```

분석기의 조건문 필터링이 안에 변수가 있을 때만 작동하고, 아예 true/false일 경우 체크를 못함

while 문 안쪽의 x에 0과 1이 대입되는 경우가 있다고 생각해, 분석기의 오차 발생

첫 번째 공격

```
while 0
  x := 1;
  while 0
    x := 0
```

분석기의 조건문 필터링이 안에 변수가 있을 때만 작동하고, 아예 true/false일 경우 체크를 못함

while 문 안쪽의 x에 0과 1이 대입되는 경우가 있다고 생각해, 분석기의 오차 발생

→ 조건문이 항상 true/false인 경우를 체크하게끔 분석기 강화

두 번째 공격

```
x := 1;
while x
  x := 0
```

두 번째 공격

```
x := 1;
while x
  x := 0
```

x에 1이 대입되었다가 0이 대입되면서, widening을 바로 적용해 $[-\infty, 1]$ 로 됨

앞으로 할 일

앞으로 할 일

- bottom-up 만으로는 좋은 공격을 찾는데 너무 오래 걸림

앞으로 할 일

- bottom-up 만으로는 좋은 공격을 찾는데 너무 오래 걸림
- “좋은 proof tree 조각” 평가 휴리스틱

앞으로 할 일

- bottom-up 만으로는 좋은 공격을 찾는데 너무 오래 걸림
- “좋은 proof tree 조각” 평가 휴리스틱
- LLM 가이드로 ‘적당히 괜찮은 증명나무 조각’ 찾기

앞으로 할 일

- bottom-up 만으로는 좋은 공격을 찾는데 너무 오래 걸림
- “좋은 proof tree 조각” 평가 휴리스틱
- LLM 가이드로 ‘적당히 괜찮은 증명나무 조각’ 찾기
- pruning 강화

앞으로 할 일

- bottom-up 만으로는 좋은 공격을 찾는데 너무 오래 걸림
- “좋은 proof tree 조각” 평가 휴리스틱
- LLM 가이드로 ‘적당히 괜찮은 증명나무 조각’ 찾기
- pruning 강화
- “비슷한 공격”들을 많이 제시해주는 건 의미가 없음. 비슷한 걸 거르고 다양하게 결과를 내주려면?

감사합니다